

Take-Home Task

“NobleOak Partners – Campaign Page + Lead Capture”

Client Summary

NobleOak Partners is a large financial consultancy working with high-net-worth individuals and family offices. Their services include wealth management, tax planning, and succession advisory.

They are rebuilding their website to better reflect their brand, improve mobile experience, and increase high-quality lead generation.

The marketing team requires a flexible, easy-to-update page structure that allows them to create and evolve campaign landing pages without heavy developer involvement.

Context

You've been asked to deliver the first slice of the new NobleOak Partners website.

The goal is to:

- Present services clearly
 - Capture high-quality leads
 - Be flexible for future marketing changes
-

Task

Build a small full-stack application that includes:

1. Content Model

Create a content model with a single “landing page” made up of reusable sections.

Minimum sections:

- Hero (headline, subheading)
- Service list (title + items)
- CTA section (text + button)
- Lead form (name, email, message)

Design the data structure so marketing teams could extend this later.

Your model should allow new section types to be added in future without requiring significant backend changes.

2. Backend API

Create an API to support:

- Fetching page content based on your data model from a mocked DB
- Submitting a lead form
- Storing submissions

You can choose:

Any stack (Node, Laravel, etc.)

Any storage (SQL, NoSQL, in-memory if justified)

You may mock persistence and external services (e.g. using in-memory storage or static JSON), but structure your code as if these were real integrations.

3. Frontend

Using a frontend framework build a simple interface (consistent with light styling) that:

- Renders the page from API data
 - Displays sections in order
 - Submits the lead form to your API
-

4. Lead Capture Flow

When a form is submitted:

- Validate input
 - Store the submission
 - Return a success/failure response
-

Constraints

Spend no more than 3–4 hours

Avoid over-engineering, focus on a thin, well-structured vertical slice rather than completeness.

We are not expecting a complete product—focus on:

- Clear structure
 - Sensible decisions
 - Working core flow
-

Deliverables

- Git repository
- README with:
 - Setup instructions
 - How you structured content (why this model?)
 - Architecture overview
 - API design decisions
 - Trade-offs you made
 - What you would improve with more time
 - Where (if anywhere) AI was used